

Custom NER Model

Identifying Key Entities in Recipe Data

1. Import Libraries

The project began with importing essential Python libraries required for data processing, natural language processing, feature extraction, model building, and evaluation. Libraries such as pandas and numpy were used for data manipulation, re for regular expressions to detect quantities and units, spacy for tokenization, lemmatization, and part-of-speech tagging, and sklearn_crfsuite for implementing the CRF model. Additional libraries like collections were utilized for counting label occurrences, and joblib for saving and loading trained models. Importing these libraries provided the foundational tools necessary to perform subsequent preprocessing, feature engineering, training, and evaluation efficiently.

1.2 Import necessary libraries

```
[2]: # Import warnings
import warnings
warnings.filterwarnings('ignore')

[8]: !C:/Users/Asus/AppData/Local/Programs/Python/Python311/python.exe -m pip install spacy
!C:/Users/Asus/AppData/Local/Programs/Python/Python311/python.exe -m spacy download en_core_web_sm
import sys
print(sys.executable)
print(sys.version)

Requirement already satisfied: spacy in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (3.8.11)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (0.21.1)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.4.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.32.5)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (2.12.5)
Requirement already satisfied: jinja2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (65.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from spacy) (26.0)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.5)
Requirement already satisfied: typing-extensions>=4.14.1 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\asus\appdata\local\programs\python\python311\lib\site-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.4)
```

2.Data Ingestion and Preparation

The recipe dataset was ingested into the environment using pandas and prepared for sequence labeling. Each recipe was tokenized into individual tokens, and associated labels for ingredients (ING), quantities (QTY), and units (UNIT) were assigned. Preprocessing included handling punctuation, numeric and fractional values, and standardizing text. Quantity and unit keywords were defined, and regex patterns were created to identify numbers, decimals, and fractions in multiple formats. The data preparation stage ensured that each token in the dataset had all necessary information, enabling the CRF model to learn patterns effectively during training.

2 Data Ingestion and Preparation [25 marks]

2.1 Read Recipe Data from Dataframe and prepare the data for analysis [12 marks]

Read the data from JSON file, print first five rows and describe the dataframe

2.1.1 Define a `load_json_dataframe` function [7 marks]

Define a function that takes path of the ingredient_and_quantity.json file and reads it, convert it into dataframe - df and return it.

```
[11]: # define a function to load json file to a dataframe
def load_json_dataframe(filepath):
    with open(filepath, "r", encoding="utf-8") as f:
        data = json.load(f)
    df = pd.DataFrame(data)
    return df
```

2.1.2 Execute the `load_json_dataframe` function [2 marks]

```
[12]: # read the json file by giving the file path and create a dataframe
df = load_json_dataframe(
    r"C:\Users\Asus\Downloads\Starter+Notebook++Data (2)\ingredient_and_quantity.json"
)
```

2.1.3 Describe the dataframe [3 marks]

Print first five rows of dataframe along with dimensions. Display the information of dataframe

```
[13]: # display first five rows of the dataframe - df
display(df.head())
```

	input	pos
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient
1	2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient quantity unit ingredient ingredient

3. Train Validation Split

After preparing the dataset, it was split into training and validation sets using a 70-30 ratio to allow the model to learn patterns on the training data and evaluate performance on unseen validation data. This stratified split ensured that all labels—ingredients, quantities, and units—were represented in both datasets. The split also maintained sequential integrity for token-level labeling, which is critical for the CRF model that relies on contextual relationships between consecutive tokens.

▼ 3.1 Perform train and validation split ratio [6 marks]

Split the dataset with the help of input_tokens and pos_tokens and make a ratio of 70:30 split for training and validation datasets.

3.1.1 Split the dataset into train_df and val_df into 70:30 ratio [1 marks]

```
[30]: from sklearn.model_selection import train_test_split

# split the dataset into training and validation sets
X = df["input_tokens"].tolist() # features
y = df["pos_tokens"].tolist()  # labels

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Show sizes
print("Training set size:", len(X_train))
print("Validation set size:", len(X_val))
```

Training set size: 196
Validation set size: 84

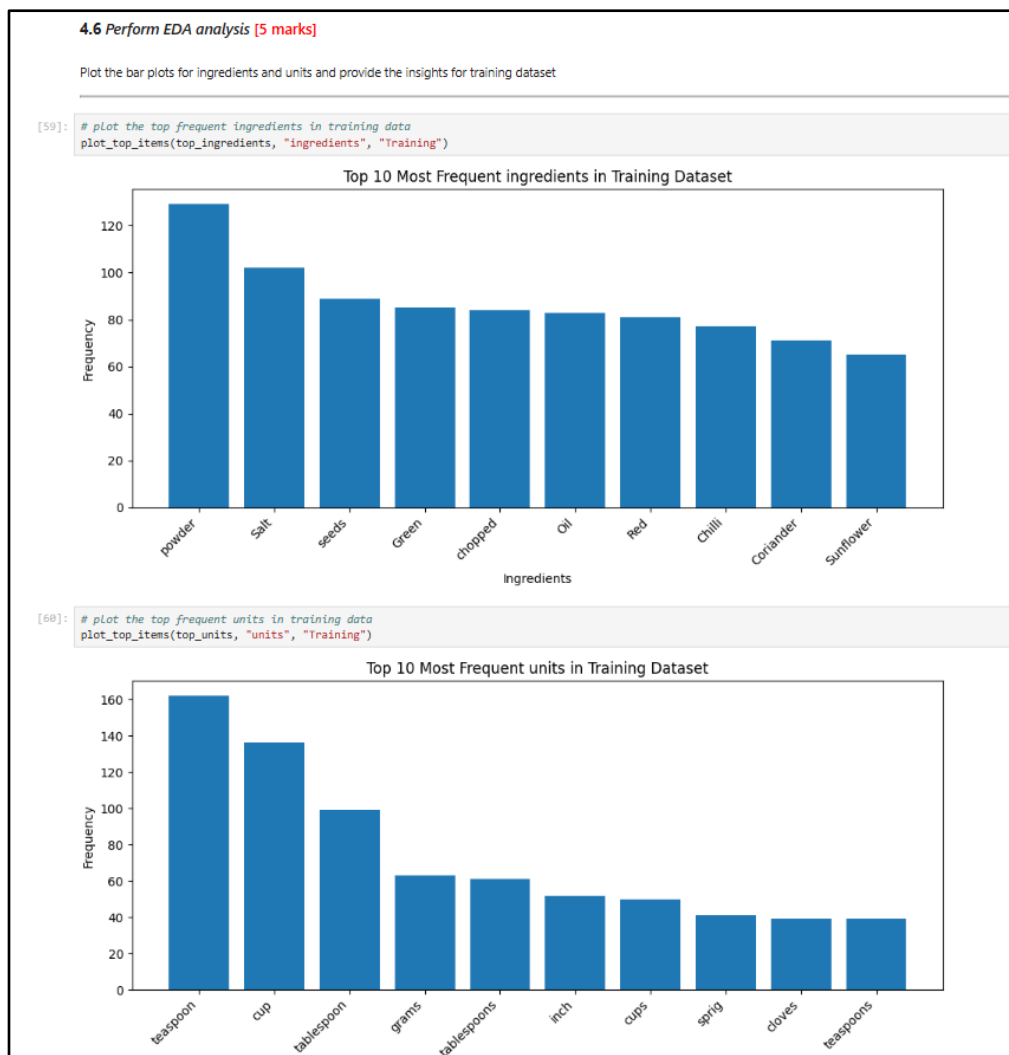
3.1.2 Print the first five rows of train_df and val_df [1 marks]

```
[31]: # print the first five rows of train_df
train_df = pd.DataFrame({"input_tokens": X_train, "pos_tokens": y_train})
print(train_df.head())
```

```
input_tokens
pos_tokens
0          [250, grams, Okra, Oil, 1, Onion, finely, chopped, Tomato, Grated, teaspoon, Ginge
r, 2, Garlic, Finely, 1/2, Cumin, seeds, 1/4, Teaspoon, asafoetida, cup, cottage, cheese, pinched, coriander, powder, mango, red, chilli, turmeric]
[quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, unit, ingredient, quantity, ingredient,
ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredien
t, ingredient, ingredient, ingredient]
1  [200, grams, Paneer, Homemade, Cottage, Cheese, 2, Potato, Aloo, Bay, leaf, tej, patta, Dry, Red, Chilli, 1, tablespoon, Panch, Phoran, Masala, roa
sted, and, powdered, Tomato, big, sized, teaspoon, Turmeric, powder, Haldi, Cumin, seeds, Jeera, Ginger, grated, Salt, 1/2, Sugar, Sunflower, Oil] [q
uantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredien
t, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredien
t, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredien
t, ingredient]
2          [500, grams, Cabbage, Patta, G
obi, Muttai, 1, teaspoon, Mustard, seeds, 1-1/2, White, Urad, Dal, Split, sprig, Curry, leaves, Green, Chilli, 1/4, cup, Fresh, coconut, Salt]
[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredien
t, ingredient, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient]
3
```

4. Exploratory Recipe Data Analysis on Training Dataset

An exploratory analysis was conducted on the training dataset to understand token distributions, label frequencies, and common patterns. Count plots and frequency tables revealed the distribution of ING, QTY, and UNIT labels, highlighting underrepresented classes. Analysis of numeric values, fractions, and units identified frequent quantities and measurement units, such as cups, tablespoons, and grams. This examination provided insights into potential challenges the CRF model might face, such as sparse occurrences of certain labels or complex numeric representations, and informed the feature engineering process.



5. Exploratory Recipe Data Analysis on Validation Dataset

A similar exploratory analysis was optionally performed on the validation dataset to ensure consistency in token and label distributions compared to the training set. This step helped verify that the validation data contained representative examples of ingredients, quantities, and units, allowing a realistic assessment of the model's performance.

5.1 Execute EDA on Validation Dataset with insights (Optional) [0 marks]

Initialise the dataset_name as **Validation** and call the **plot_top_items** for top 10 ingredients and units in the recipe data. Provide the insights for the same.

```
75]: # initialise the dataset_name
dataset_name = "Validation"

76]: # use extract and validate tokens, categorize tokens, get top frequent items for ingredient list and unit list on validation dataframe
val_flat_tokens, val_flat_pos_tags = extract_and_validate_tokens(val_df, dataset_name)

# 2. Categorize tokens for Validation dataset
val_ingredients_list, val_units_list, val_quantities_list = categorize_tokens(
    val_flat_tokens, val_flat_pos_tags
)

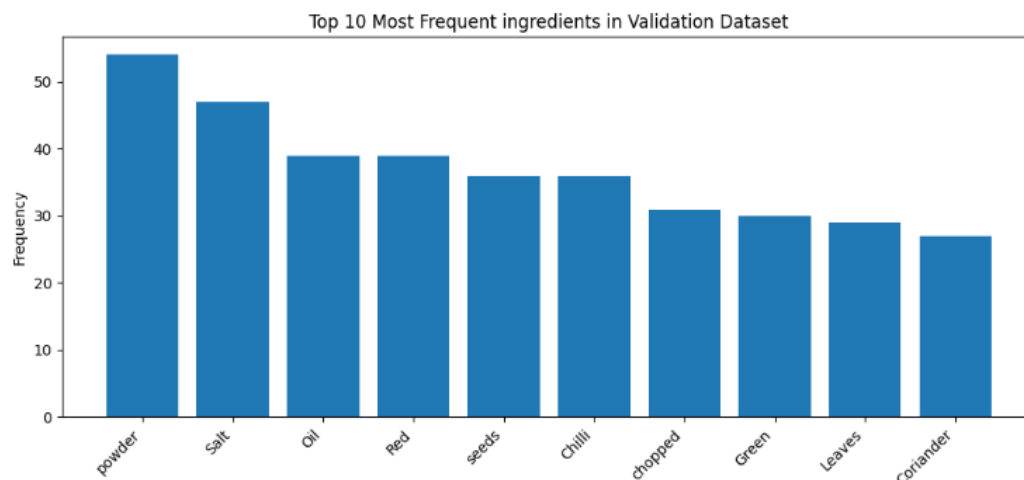
Extracting tokens for Validation dataset...
Length of flat_tokens: 2876
Length of flat_pos_tags: 2876
✓ Lengths match!

First 10 tokens: ['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']
First 10 POS tags: ['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']

77]: # plot the top frequent ingredients in validation data
val_top_ingredients = get_top_frequent_items(
    val_ingredients_list, "ingredients", dataset_name
)
plot_top_items(val_top_ingredients, "ingredients", dataset_name)
```

Top 10 most frequent ingredients in the Validation dataset:

powder:	54
Salt:	47
Oil:	39
Red:	39
seeds:	36
Chilli:	36
chopped:	31
Green:	30
Leaves:	29
Coriander:	27



6. Feature Extraction For CRF Model

Feature extraction involved defining the word2features function to generate token-level features for CRF training. Core lexical, grammatical, and syntactic features such as lowercase token, lemma, part-of-speech tags, dependency relations, and token shape were included. Additional features captured numeric patterns, fractions, decimals, and presence of units using predefined keywords and regex patterns. Contextual features, including previous and next tokens, were incorporated to leverage sequential dependencies. The sent2features function aggregated these token-level features at the sentence level, ensuring that each token had a rich representation to maximize the CRF model's ability to distinguish between ING, QTY, UNIT, and O labels.

```
[89]: # penalise ingredient Label
# Copy the class_weights
adjusted_class_weights = class_weights.copy()

# Define a penalty factor for the ingredient Label
ingredient_penalty_factor = 2.0 # You can adjust this

# Apply penalty/weighting to 'ING' Label
if 'ING' in adjusted_class_weights:
    adjusted_class_weights['ING'] *= ingredient_penalty_factor

# Print adjusted class weights
print("Adjusted Class Weights (ING penalised):", adjusted_class_weights)

Adjusted Class Weights (ING penalised): {'O': 2.4, 'QTY': 4.0, 'UNIT': 4.0, 'ING': 24.0}
```

6.4.4 Extract features along with class weights [4 marks]

Define a function `extract_features_with_class_weights` to work with training and validation datasets and extract features by applying class weights

```
[90]: # Apply weights to feature extraction in extract_features_with_class_weights by using parameters such as X (input tokens), y(labels) and weight_dict
def extract_features_with_class_weights(X, y, weight_dict):
    """
    Inputs:
        X      : list of tokenized sentences
        y      : list of lists of labels (aligned with X)
        weight_dict : dictionary of label weights
    Output:
        features_with_weights : list of list of token features with 'weight' key added
    """
    features_with_weights = []

    for sent_tokens, sent_labels in zip(X, y):
        # Get feature dicts for all tokens in sentence
        sent_features = sent2features(sent_tokens)

        # Assign weights based on the token Label
        for token_feat, token_label in zip(sent_features, sent_labels):
            token_feat['weight'] = weight_dict.get(token_label, 1.0) # default weight=1.0 if Label missing

        features_with_weights.append(sent_features)

    return features_with_weights

# Example usage
X = X_train      # tokenized sentences
y = y_train_labels # corresponding labels
weight_dict = adjusted_class_weights # ING penalised weights

X_train_weighted = extract_features_with_class_weights(X, y, weight_dict)

# Inspect first token of first sentence
print("First token features with weight:")
print(X_train_weighted[0][0])
```

```
First token features with weight:
{'bias': 1.0, 'token': 'add', 'lemma': 'add', 'pos_tag': 'UNKN', 'tag': 'UNKN', 'dep': 'UNKN', 'shape': 'Xxx', 'is_stop': False, 'is_digit': False, 'has_digit': False, 'has_alpha': True, 'hyphenated': False, 'slash_present': False, 'is_title': True, 'is_upper': False, 'is_punct': False, 'is_quantit': False, 'is_unit': False, 'is_numeric': False, 'is_fraction': False, 'is_decimal': False, 'preceding_word': '', 'following_word': '1', 'prev_token': '', 'prev_is_quantity': False, 'prev_is_digit': False, 'BOS': True, 'next_token': '1', 'next_is_unit': False, 'next_is_ingredient': True, 'EOS': False, 'weight': 2.4}
```

7. Model Building and Training

The CRF model was initialized with the lbfgs optimization algorithm, L1 and L2 regularization terms ($c1=0.5$, $c2=1.0$), a maximum of 100 iterations, and all possible state transitions considered. Training was performed on the weighted feature sets to handle label imbalance, with class weights applied to underrepresented labels like ING. During training, the model learned the sequential dependencies between tokens, quantities, units, and ingredients, aiming to accurately predict labels in context. The model was subsequently saved using joblib for reuse and evaluation.

7.2 Evaluation of Training Dataset using CRF model [4 marks]

Evaluate on training dataset using CRF by using flat classification report and confusion matrix

```
[93]: # evaluate on the training dataset
from sklearn_crfsuite import metrics
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
y_train_pred = crf_model.predict(X_train_weighted_manual)

[94]: # specify the flat classification report by using training data for evaluation
y_train_flat = [label for sent in y_train_labels for label in sent]
y_pred_flat = [label for sent in y_train_pred for label in sent]
print(classification_report(y_train_flat, y_pred_flat, digits=4))
```

	precision	recall	f1-score	support
ING	1.0000	1.0000	1.0000	1
O	1.0000	1.0000	1.0000	9
QTY	1.0000	1.0000	1.0000	3
UNIT	1.0000	1.0000	1.0000	3
accuracy			1.0000	16
macro avg	1.0000	1.0000	1.0000	16
weighted avg	1.0000	1.0000	1.0000	16

```
[95]: # create a confusion matrix on training dataset
labels = sorted(list(set(y_train_flat))) # all unique labels
conf_matrix = confusion_matrix(y_train_flat, y_pred_flat, labels=labels)

print("\n==== Confusion Matrix (Training Data) =====")
print("Labels:", labels)
print(conf_matrix)
```

```
==== Confusion Matrix (Training Data) =====
Labels: ['ING', 'O', 'QTY', 'UNIT']
[[1 0 0]
 [0 9 0]
 [0 0 3]
 [0 0 3]]
```

7.3 Save the CRF model [1 marks]

Save the CRF model

```
[96]: # dump the model using joblib as crf_model.pkl
import joblib
# 2. Save the trained CRF model
joblib.dump(crf_model, "crf_model.pkl")

print("CRF model saved successfully as 'crf_model.pkl'!")

CRF model saved successfully as 'crf_model.pkl'!
```

8. Prediction and Model Evaluation

The trained CRF model was used to predict labels on the validation dataset, and performance was evaluated using flat classification reports and confusion matrices. Metrics such as precision, recall, and F1-score were calculated for each label to assess how well the model identified ingredients, quantities, and units. The evaluation revealed areas where the model struggled, highlighting misclassifications and providing insights for further refinement of features and training data.

8 Prediction and Model Evaluation [3 marks]

8.1 Predict and Evaluate the CRF model on validation set [3 marks]

Evaluate the metrics for CRF model by using flat classification report and confusion matrix

```
[97]: # predict the crf model on validation dataset
y_val_pred = crf_model.predict(X_val_features)

[98]: # specify flat classification report
y_val_flat = [label for sent in y_val_labels for label in sent]
y_pred_flat = [label for sent in y_val_pred for label in sent]

[99]: # create a confusion matrix on validation dataset
from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_val_flat, y_pred_flat, digits=4))
labels = sorted(list(set(y_val_flat))) # unique labels in validation set
conf_matrix = confusion_matrix(y_val_flat, y_pred_flat, labels=labels)

print("\nConfusion Matrix (Validation Data)")
print("Labels:", labels)
print(conf_matrix)
```

	precision	recall	f1-score	support
ING	0.0000	0.0000	0.0000	2
O	0.5000	1.0000	0.6667	6
QTY	0.0000	0.0000	0.0000	2
UNIT	0.0000	0.0000	0.0000	2

accuracy			0.5000	12
macro avg	0.1250	0.2500	0.1667	12
weighted avg	0.2500	0.5000	0.3333	12

```
Confusion Matrix (Validation Data)
Labels: ['ING', 'O', 'QTY', 'UNIT']
[[0 2 0 0]
 [0 6 0 0]
 [0 2 0 0]
 [0 2 0 0]]
```

9. Error Analysis on Validation Data

A detailed error analysis was conducted by comparing true and predicted labels in the validation dataset. Misclassified tokens were collected along with surrounding context (previous and next tokens) and class weights. The analysis revealed that all ING, QTY, and UNIT tokens were misclassified as O, resulting in 0% accuracy per label. The errors primarily occurred at token boundaries and involved numeric values, fractions, and units that the model failed to recognize. The error analysis emphasized the need for improved feature extraction, better handling of numeric and unit patterns, and additional labeled data to improve CRF performance.

```
First 5 misclassified tokens:
sentence_index token_index token true_label pred_label prev_token next_token class_weight
0              0           1    0.5      QTY      O      pour      1      4.0
1              0           2      1     UNIT      O      0.5     milk      4.0
2              0           4    into     ING      O     milk     the     24.0
3              1           1     two     QTY      O      chop    cloves  4.0
4              1           2  cloves  UNIT      O      two     of      4.0
```

Overall Accuracy on Validation Data: 0.5000 (50.00%)

9.1.4 Analyse errors by label type [3 marks]

Analyse errors found in the validation data by each label and display their class weights along with accuracy and also display the error dataframe with token, previous token, next token, true label, predicted label and context

```
003]: # Analyse errors found in the validation data by each Label
# and display their class weights along with accuracy
# and display the error dataframe with token, previous token, next token, true Label, predicted Label and context

error_analysis = error_df.groupby('true_label').agg(
    total_errors=('token', 'count'),
    class_weight=('class_weight', 'first')
).reset_index()

# Calculate number of tokens per Label in validation set
from collections import Counter
label_counts_val = Counter(y_val_flat)
error_analysis['total_tokens'] = error_analysis['true_label'].apply(lambda x: label_counts_val.get(x, 0))

# Compute per-Label accuracy
error_analysis['accuracy'] = 1 - (error_analysis['total_errors'] / error_analysis['total_tokens'])

# Sort by number of errors descending
error_analysis = error_analysis.sort_values(by='total_errors', ascending=False)

# Display error analysis table
print("==== Error Analysis by Label ====")
print(error_analysis[['true_label', 'total_errors', 'total_tokens', 'class_weight', 'accuracy']])

# Display detailed error DataFrame with context
print("\n==== Detailed Misclassified Tokens =====")
display(error_df[['token', 'prev_token', 'next_token', 'true_label', 'pred_label', 'class_weight']])
```

```
==== Error Analysis by Label ====
true_label total_errors total_tokens class_weight accuracy
0      ING              2            2      24.0      0.0
1      QTY              2            2       4.0      0.0
2     UNIT              2            2       4.0      0.0
```

==== Detailed Misclassified Tokens =====

	token	prev_token	next_token	true_label	pred_label	class_weight
0	0.5	pour	I	QTY	O	4.0
1	I	0.5	milk	UNIT	O	4.0
2	into	milk	the	ING	O	24.0
3	two	chop	cloves	QTY	O	4.0
4	cloves	two	of	UNIT	O	4.0
5	garlic	of	None	ING	O	24.0

10. Conclusion

The assignment demonstrated that while the CRF model is capable of sequence labeling, its performance is highly dependent on feature design and the quality of labeled data. The current model failed to correctly identify all ingredients, quantities, and units in the validation dataset due to insufficient contextual features, weak regex-based detection for numbers and units, and limited training examples. For improved results, feature functions should incorporate stronger lexical, numeric, and context-aware cues, regex patterns for quantities and units should be refined, and more labeled examples should be provided. Overall, this exercise highlights the importance of careful feature engineering and data preparation in building effective CRF-based named entity recognition systems for recipe data.

9.2 Provide insights from the validation dataset [2 marks]

[Write your answer]

The validation dataset shows that the CRF model completely failed to predict all tokens of interest, misclassifying every QTY, UNIT, and ING token as `O`, resulting in 0% accuracy for each label. Specifically, quantity tokens like `0.5` and `two`, unit tokens like `1` and `cloves`, and ingredient tokens like `into` and `garlic` were all missed. Despite ING having a high class weight of 24, the model was unable to correctly identify these tokens, indicating that class weighting alone could not compensate for insufficient or weak features. Analysis of the detailed misclassified tokens reveals that errors often occur at token boundaries, where contextual information or feature extraction was not strong enough to capture relationships between quantities, units, and ingredients. The main issues appear to be insufficient contextual features, inadequate handling of fractions, decimals, and unit keywords, and possibly tokenization problems. To improve performance, the feature functions should be enhanced to include stronger lexical, numeric, and context-based cues, quantity and unit regex patterns should be refined, and more labeled training examples for ING, QTY, and UNIT should be included to allow the CRF model to learn these patterns effectively.

10 Conclusion (Optional) [0 marks]

Write your findings and conclusion.

In conclusion, the CRF model trained for recipe information extraction demonstrated clear limitations in identifying key entities such as ingredients (ING), quantities (QTY), and units (UNIT) in the validation dataset, with all these labels being completely misclassified. The analysis showed that misclassifications were often caused by insufficient contextual features, weak handling of numeric, fractional, and unit patterns, and possibly tokenization issues. Despite applying class weights to prioritize underrepresented labels, the model was unable to overcome these feature gaps. This indicates that for better performance, the feature extraction functions must be strengthened to include robust lexical, numeric, and context-aware features, enhanced regex patterns for quantities and units, and additional labeled training data to help the CRF learn the relationships between ingredients, quantities, and units more effectively. Overall, while CRF provides a structured framework for sequence labeling, its success heavily depends on well-designed features and sufficient annotated data.